Quantium Retail Analytics (Task 2): Trial Store Evaluation

Stores Under Trial: 77, 86, 88
Category: Chips
Client: Julia (via Zilinka)
Trial Period: February – April 2019
Pre-Trial Period: July 2018 – January 2019

> This notebook evaluates the performance of a new in-store layout trial using Pearson correlation and magnitude-distance scoring to select control stores, followed by 95% CI-based significance testing.

1. Imports & Configuration

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline
plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'font.size': 10
})

# constants
TRIAL_STORES = [77, 86, 88]
TRIAL_START  = pd.Period('2019-02', freq='M')
TRIAL_END    = pd.Period('2019-04', freq='M')

# colour palette
BLUE   = '#1F77B4'
ORANGE = '#FF7F0E'
GREEN  = '#2CA02C'
RED    = '#D62728'
```

2. Data Loading & Preparation

```
df = pd.read_csv('QVI_data.csv', parse_dates=['DATE'])
print(f"Shape: {df.shape}")
print(f"Date range: {df['DATE'].min().date()} →
{df['DATE'].max().date()}")
df.head()
```

```
Shape: (264834, 12)
Date range: 2018-07-01 → 2019-06-30

   LYLTY_CARD_NBR       DATE  STORE_NBR  TXN_ID  PROD_NBR  \
0            1000 2018-10-17          1       1         5
1            1002 2018-09-16          1       2        58
2            1003 2019-03-07          1       3        52
3            1003 2019-03-08          1       4       106
4            1004 2018-11-02          1       5        96

                                PROD_NAME  PROD_QTY  TOT_SALES
PACK_SIZE  \
0  Natural Chip        Compny SeaSalt175g         2        6.0
175
1   Red Rock Deli Chikn&Garlic Aioli 150g         1        2.7
150
2   Grain Waves Sour    Cream&Chives 210G         1        3.6
210
3  Natural ChipCo      Hony Soy Chckn175g         1        3.0
175
4         WW Original Stacked Chips 160g         1        1.9
160

        BRAND              LIFESTAGE PREMIUM_CUSTOMER
0     NATURAL  YOUNG SINGLES/COUPLES          Premium
1         RRD  YOUNG SINGLES/COUPLES       Mainstream
2     GRNWVES         YOUNG FAMILIES           Budget
3     NATURAL         YOUNG FAMILIES           Budget
4  WOOLWORTHS  OLDER SINGLES/COUPLES       Mainstream
```

```
df['MONTH'] = df['DATE'].dt.to_period('M')

# Monthly aggregations per store
monthly = (
    df.groupby(['STORE_NBR', 'MONTH'])
      .agg(
          total_sales   = ('TOT_SALES',      'sum'),
          n_customers   = ('LYLTY_CARD_NBR', 'nunique'),
          n_txns        = ('TXN_ID',          'nunique')
      )
      .reset_index()
)
monthly['txns_per_cust'] = monthly['n_txns'] / monthly['n_customers']
```

```
print(f"Monthly records: {len(monthly)}")
monthly.head(8)
```

```
Monthly records: 3169
```

	STORE_NBR	MONTH	total_sales	n_customers	n_txns	txns_per_cust
0	1	2018-07	206.9	49	52	1.061224
1	1	2018-08	176.1	42	43	1.023810
2	1	2018-09	278.8	59	62	1.050847
3	1	2018-10	188.1	44	45	1.022727
4	1	2018-11	192.6	46	47	1.021739
5	1	2018-12	189.6	42	47	1.119048
6	1	2019-01	154.8	35	36	1.028571
7	1	2019-02	225.4	52	55	1.057692

```
# Splitting pre-trial (for control matching) vs full dataset
pre = monthly[monthly['MONTH'] < TRIAL_START].copy()
print(f"Pre-trial months: {sorted(pre['MONTH'].unique())}")
print(f"Stores in dataset: {monthly['STORE_NBR'].nunique()}")
```

```
Pre-trial months: [Period('2018-07', 'M'), Period('2018-08', 'M'),
Period('2018-09', 'M'), Period('2018-10', 'M'), Period('2018-11',
'M'), Period('2018-12', 'M'), Period('2019-01', 'M')]
Stores in dataset: 272
```

3. Control Store Selection

We combine two measures to score each candidate store against a trial store:

Measure	What it captures
Pearson Correlation	Similarity in *trend* (shape of the time series)
Magnitude Score	Similarity in *level* — `1 − (dist − min)/(max − min)`

Final composite = **0.5 × Pearson + 0.5 × Magnitude**
Ranks are summed across all three metrics; the store with the lowest cumulative rank is selected.

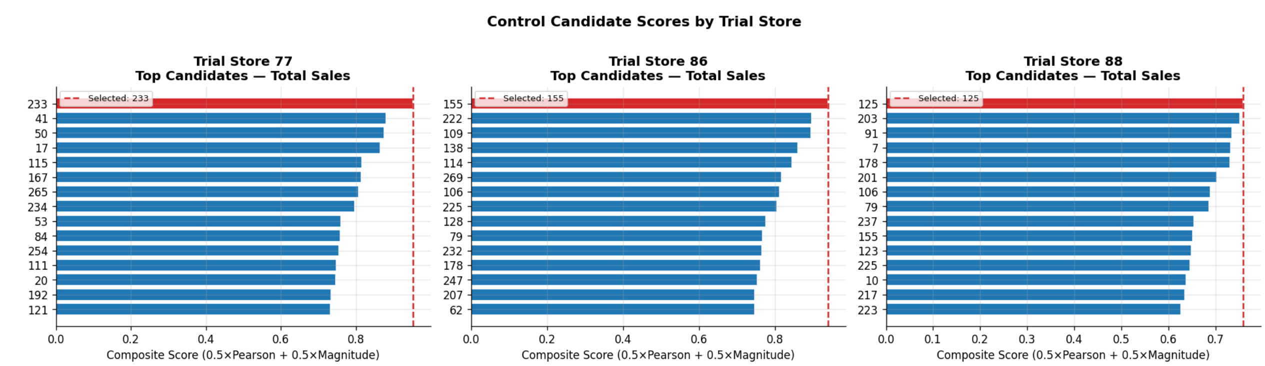
```
def magnitude_score(series):
    """Normalising distances so that the smallest distance → score of 1."""
    mn, mx = series.min(), series.max()
    if mx == mn:
        return pd.Series(np.ones(len(series)), index=series.index)
    return 1 - (series - mn) / (mx - mn)


def score_candidates(trial_store, metric='total_sales', n=15):
```

```
    """
    Scoring all non-trial stores against `trial_store` on a single
metric.
    Returns a DataFrame sorted by composite score (descending).
    """
    candidates = [
        s for s in pre['STORE_NBR'].unique()
        if s not in TRIAL_STORES and s != trial_store
    ]
    t_series = pre[pre['STORE_NBR'] == trial_store].set_index('MONTH')
[metric]

    records = []
    for s in candidates:
        c_series = pre[pre['STORE_NBR'] == s].set_index('MONTH')
[metric]
        common   = t_series.index.intersection(c_series.index)
        if len(common) < 6:
            continue
        r, _  = stats.pearsonr(t_series[common], c_series[common])
        dist  = np.mean(np.abs(t_series[common].values -
c_series[common].values))
        records.append({'store': s, 'pearson': r, 'dist': dist})

    res        = pd.DataFrame(records)
    res['mag'] = magnitude_score(res['dist'])
    res['score'] = 0.5 * res['pearson'] + 0.5 * res['mag']
    return res.sort_values('score',
ascending=False).head(n).reset_index(drop=True)


def select_best_control(trial_store):
    """
    Picking the best overall control by summing ranks across all three
metrics.
    Lower cumulative rank = better overall match.
    """
    cum_rank = {}
    for metric in ['total_sales', 'n_customers', 'txns_per_cust']:
        top = score_candidates(trial_store, metric, n=30)
        for rank, row in top.iterrows():
            s = row['store']
            cum_rank[s] = cum_rank.get(s, 0) + rank
    return int(min(cum_rank, key=cum_rank.get))


# running selection
controls = {t: select_best_control(t) for t in TRIAL_STORES}
print("Selected control stores:")
```

```
for trial, ctrl in controls.items():
    print(f"  Trial {trial}  →  Control {ctrl}")

Selected control stores:
  Trial 77  →  Control 233
  Trial 86  →  Control 155
  Trial 88  →  Control 125
```

3.1 Candidate Score Charts

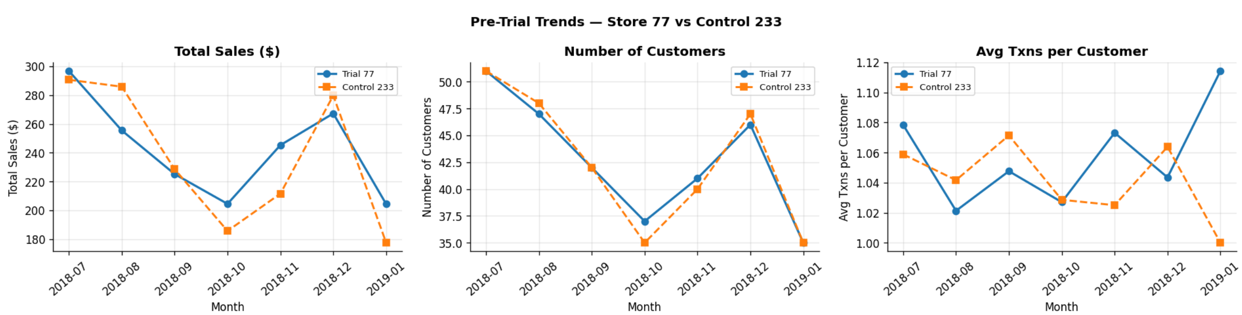
```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

for ax, trial in zip(axes, TRIAL_STORES):
    ctrl = controls[trial]
    top  = score_candidates(trial, 'total_sales', n=15)

    bar_colors = [RED if row['store'] == ctrl else BLUE for _, row in 
top.iterrows()]
    ax.barh(top['store'].astype(str), top['score'], color=bar_colors, 
edgecolor='white')
    ax.set_xlabel('Composite Score (0.5×Pearson + 0.5×Magnitude)')
    ax.set_title(f'Trial Store {trial}\nTop Candidates — Total Sales', 
fontweight='bold')
    ax.invert_yaxis()
    ax.axvline(top[top['store']==ctrl]['score'].values[0] if ctrl in 
top['store'].values else 0,
               color=RED, lw=1.5, ls='--', label=f'Selected: {ctrl}')
    ax.legend(fontsize=8)

plt.suptitle('Control Candidate Scores by Trial Store', fontsize=13, 
fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```

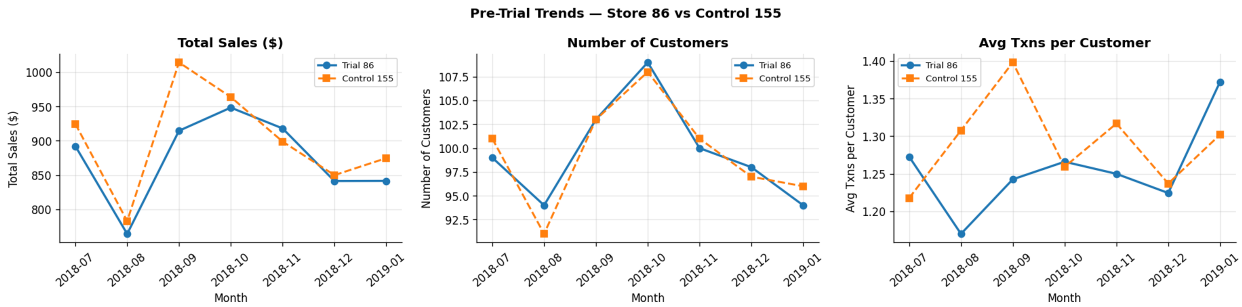


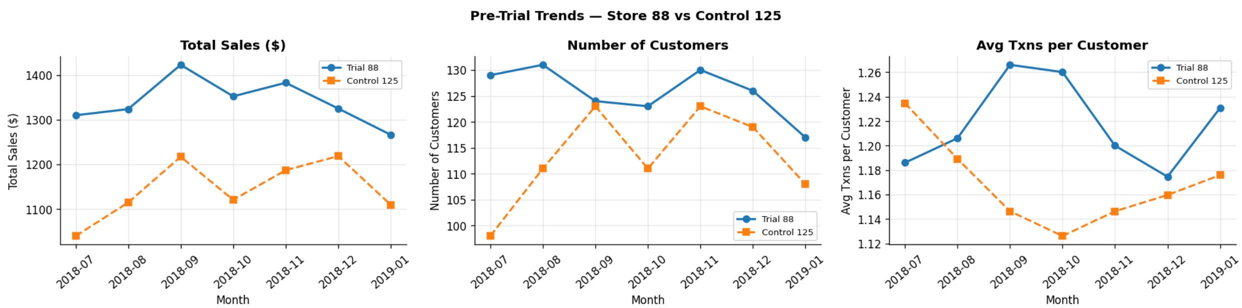
4. Pre-Trial Trend Validation

Before accepting a control store we confirm that the pre-trial trends move together.
A control store with a divergent trajectory would produce a misleading counterfactual.

```python
def plot_pre_trends(trial, ctrl):
    """Side-by-side pre-trial trend plots for a trial-control pair."""
    metrics = [
        ('total_sales',    'Total Sales ($)'),
        ('n_customers',    'Number of Customers'),
        ('txns_per_cust', 'Avg Txns per Customer'),
    ]
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))

    for ax, (metric, ylabel) in zip(axes, metrics):
        t = pre[pre['STORE_NBR'] == trial].set_index('MONTH')[metric]
        c = pre[pre['STORE_NBR'] == ctrl ].set_index('MONTH')[metric]
        common = sorted(t.index.intersection(c.index))
        xs = [str(m) for m in common]

        ax.plot(xs, t[common].values, 'o-', color=BLUE,   lw=2, label=f'Trial {trial}')
        ax.plot(xs, c[common].values, 's--', color=ORANGE, lw=1.8, label=f'Control {ctrl}')
        ax.set_xlabel('Month')
        ax.set_ylabel(ylabel)
        ax.set_title(ylabel, fontweight='bold')
        ax.tick_params(axis='x', rotation=40)
        ax.legend(fontsize=8)

    fig.suptitle(f'Pre-Trial Trends — Store {trial} vs Control {ctrl}',
                 fontsize=12, fontweight='bold')
    plt.tight_layout()
    plt.show()


for trial in TRIAL_STORES:
    plot_pre_trends(trial, controls[trial])
```







5. Trial Period Assessment

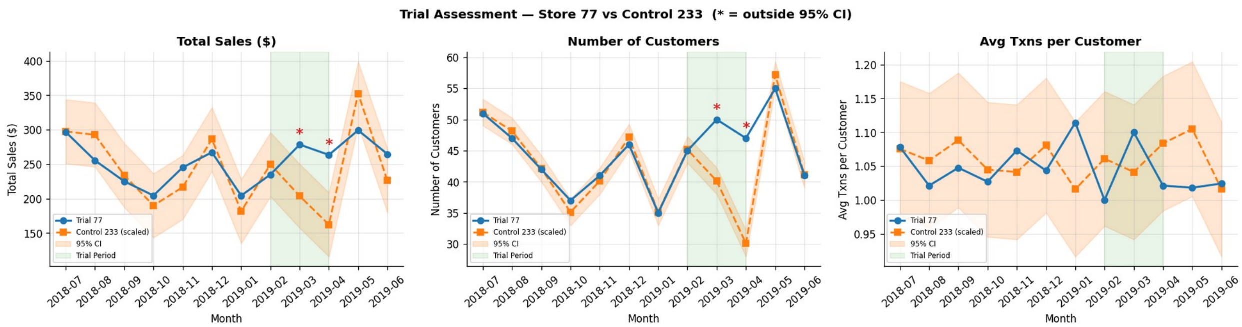
For each trial–control pair:

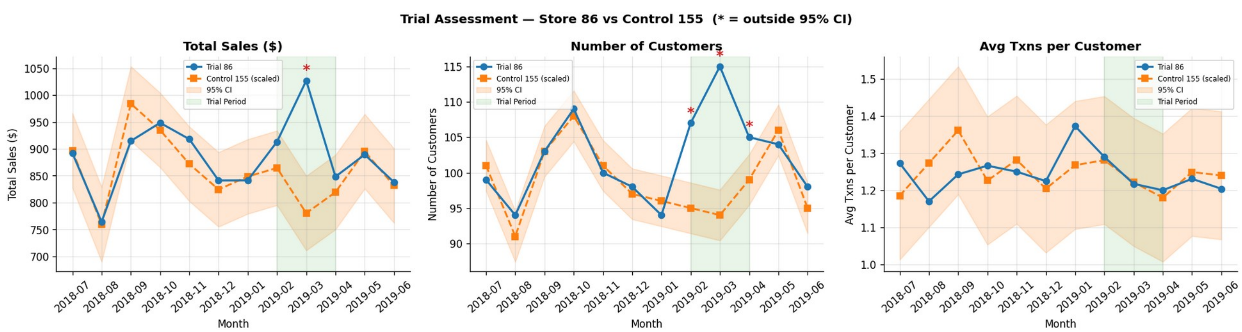
1. **Scale** the control series to the trial's pre-trial mean level (removes permanent level differences).
2. **Build a 95% CI** from the standard deviation of pre-trial residuals × 1.96.
3. **Flag** trial-period months where the trial store falls outside the CI — indicating statistically significant deviation.

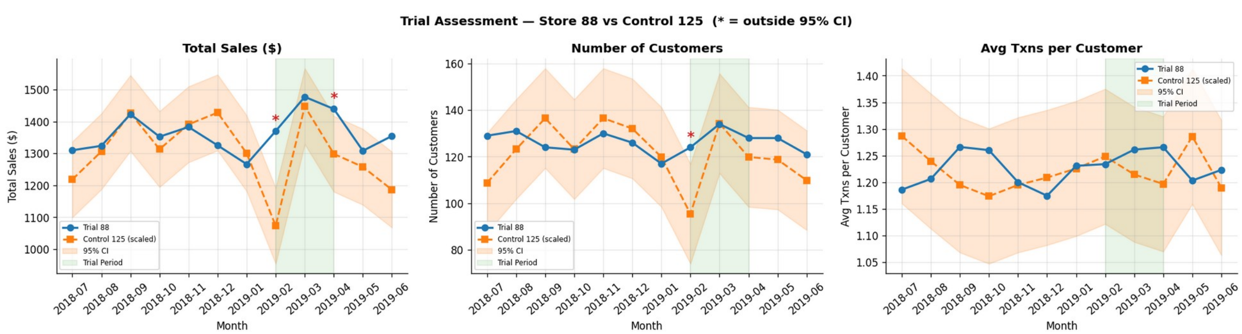
```
def scale_factor(trial, ctrl, metric):
    """Ratio of pre-trial means — scales control to trial's level."""
    t_mean = pre[pre['STORE_NBR'] == trial][metric].mean()
    c_mean = pre[pre['STORE_NBR'] == ctrl ][metric].mean()
    return t_mean / c_mean if c_mean else 1


def ci_half_width(trial, ctrl, metric):
    """95% CI half-width based on pre-trial residuals."""
    t = pre[pre['STORE_NBR'] == trial].set_index('MONTH')[metric]
    c = pre[pre['STORE_NBR'] == ctrl ].set_index('MONTH')[metric]
    sf     = scale_factor(trial, ctrl, metric)
    common = t.index.intersection(c.index)
    resid  = t[common].values - c[common].values * sf
    return 1.96 * np.std(resid, ddof=1)


def plot_trial_assessment(trial, ctrl):
    """Full-year plot with CI band and trial-period shading."""
    metrics = [
```

```
        ('total_sales',    'Total Sales ($)'),
        ('n_customers',    'Number of Customers'),
        ('txns_per_cust', 'Avg Txns per Customer'),
    ]
    fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))

    for ax, (metric, ylabel) in zip(axes, metrics):
        sf = scale_factor(trial, ctrl, metric)
        hw = ci_half_width(trial, ctrl, metric)

        t_all = monthly[monthly['STORE_NBR'] ==
trial].set_index('MONTH')[metric]
        c_all = monthly[monthly['STORE_NBR'] ==
ctrl ].set_index('MONTH')[metric]
        common   = sorted(t_all.index.intersection(c_all.index))
        xs       = [str(m) for m in common]
        t_vals   = t_all[common].values
        c_scaled = c_all[common].values * sf

        ax.plot(xs, t_vals,   'o-',  color=BLUE,   lw=2,
label=f'Trial {trial}',           zorder=3)
        ax.plot(xs, c_scaled, 's--', color=ORANGE,  lw=1.8,
label=f'Control {ctrl} (scaled)', zorder=2)
        ax.fill_between(xs, c_scaled - hw, c_scaled + hw,
                        alpha=0.18, color=ORANGE, label='95% CI')

        # shade trial window
        trial_xs = [str(m) for m in common if TRIAL_START <= m <=
TRIAL_END]
        if trial_xs:
            ax.axvspan(trial_xs[0], trial_xs[-1],
                       alpha=0.10, color=GREEN, label='Trial Period')

        # significance markers
        for i, m in enumerate(common):
            if TRIAL_START <= m <= TRIAL_END:
                if t_vals[i] > c_scaled[i] + hw or t_vals[i] <
c_scaled[i] - hw:
                    ax.annotate('*', (str(m), t_vals[i]),
                                textcoords='offset points', xytext=(0,
6),
                                ha='center', fontsize=14, color=RED,
fontweight='bold')

        ax.set_xlabel('Month')
        ax.set_ylabel(ylabel)
        ax.set_title(ylabel, fontweight='bold')
        ax.tick_params(axis='x', rotation=40)
        ax.legend(fontsize=7)
```

```
    fig.suptitle(f'Trial Assessment — Store {trial} vs Control {ctrl}
'
                 f'(* = outside 95% CI)',
                 fontsize=12, fontweight='bold')
    plt.tight_layout()
    plt.show()


for trial in TRIAL_STORES:
    plot_trial_assessment(trial, controls[trial])
```







6. Statistical Significance: Paired t-Tests

```
def run_ttests(trial, ctrl):
    """Paired t-test on trial vs scaled-control during the trial
months."""
    results = []
```

```
    for metric, label in [('total_sales',   'Total Sales'),
                           ('n_customers',   'Customers'),
                           ('txns_per_cust', 'Txns/Customer')]:
        sf   = scale_factor(trial, ctrl, metric)
        mask = (monthly['MONTH'] >= TRIAL_START) & (monthly['MONTH']
<= TRIAL_END)
        t_v  = monthly[(monthly['STORE_NBR'] == trial) & mask]
[metric].values
        c_v  = monthly[(monthly['STORE_NBR'] == ctrl)  & mask]
[metric].values * sf

        if len(t_v) >= 2 and len(c_v) >= 2:
            t_stat, p = stats.ttest_rel(t_v, c_v) if len(t_v) ==
len(c_v) \
                        else stats.ttest_ind(t_v, c_v)
        else:
            t_stat, p = np.nan, np.nan

        results.append({
            'Metric':       label,
            't-statistic':  round(t_stat, 3),
            'p-value':      round(p, 4),
            'Significant?': 'Yes ✓' if (not np.isnan(p) and p < 0.05)
else 'No'
        })
    return pd.DataFrame(results)


for trial in TRIAL_STORES:
    ctrl = controls[trial]
    print(f"\n{'='*55}")
    print(f"  Trial Store {trial}  vs  Control Store {ctrl}")
    print(f"{'='*55}")
    display(run_ttests(trial, ctrl))
```

```
=======================================================
  Trial Store 77  vs  Control Store 233
=======================================================
```

	Metric	t-statistic	p-value	Significant?
0	Total Sales	1.531	0.2654	No
1	Customers	1.793	0.2148	No
2	Txns/Customer	-0.538	0.6447	No

```
=======================================================
  Trial Store 86  vs  Control Store 155
=======================================================
```

```
           Metric  t-statistic  p-value Significant?
0    Total Sales        1.555   0.2602           No
1      Customers        2.982   0.0964           No
2  Txns/Customer        1.144   0.3711           No


=======================================================
  Trial Store 88  vs  Control Store 125
=======================================================

           Metric  t-statistic  p-value Significant?
0    Total Sales        2.008   0.1824           No
1      Customers        1.419   0.2918           No
2  Txns/Customer        1.354   0.3084           No
```

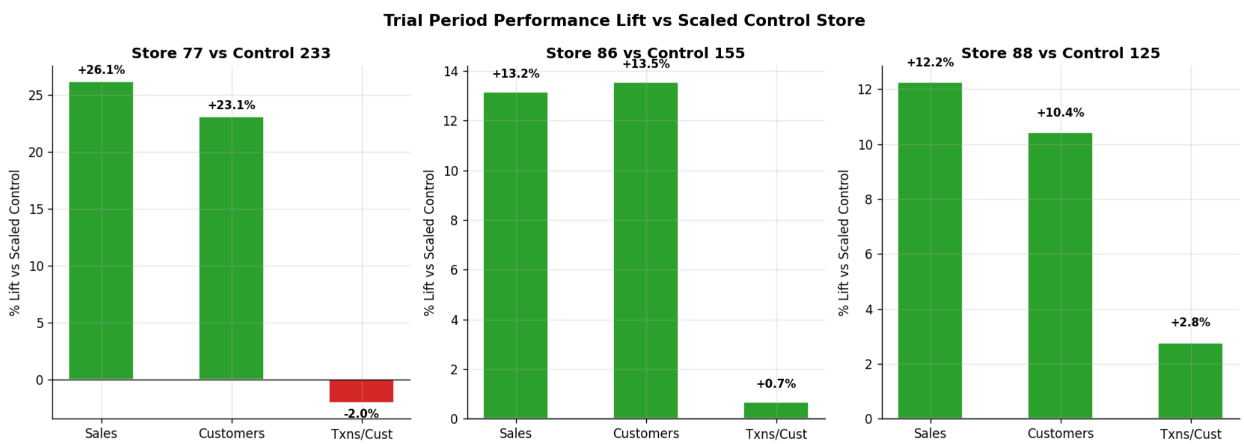
7. Summary Lift & Recommendations

```
summary_rows = []
for trial in TRIAL_STORES:
    ctrl = controls[trial]
    for metric, label in [('total_sales',   'Sales'),
                           ('n_customers',   'Customers'),
                           ('txns_per_cust', 'Txns/Cust')]:
        sf     = scale_factor(trial, ctrl, metric)
        mask   = (monthly['MONTH'] >= TRIAL_START) & (monthly['MONTH']
<= TRIAL_END)
        t_sum  = monthly[(monthly['STORE_NBR'] == trial) & mask]
[metric].sum()
        c_sum  = monthly[(monthly['STORE_NBR'] == ctrl)  & mask]
[metric].sum() * sf
        lift   = (t_sum - c_sum) / c_sum * 100 if c_sum else 0
        summary_rows.append({'Trial': trial, 'Control': ctrl,
'Metric': label, 'Lift (%)': round(lift, 2)})

summary = pd.DataFrame(summary_rows)
display(summary)
```

	Trial	Control	Metric	Lift (%)
0	77	233	Sales	26.15
1	77	233	Customers	23.07
2	77	233	Txns/Cust	-2.03
3	86	155	Sales	13.15
4	86	155	Customers	13.54
5	86	155	Txns/Cust	0.66
6	88	125	Sales	12.24
7	88	125	Customers	10.42
8	88	125	Txns/Cust	2.77

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
```

```
for ax, (trial, grp) in zip(axes, summary.groupby('Trial')):
    ctrl   = controls[trial]
    colors = [GREEN if v >= 0 else RED for v in grp['Lift (%)']]
    bars   = ax.bar(grp['Metric'], grp['Lift (%)'], color=colors,
edgecolor='white', width=0.5)
    ax.axhline(0, color='black', lw=0.8)
    ax.set_title(f'Store {trial} vs Control {ctrl}',
fontweight='bold')
    ax.set_ylabel('% Lift vs Scaled Control')
    for bar, v in zip(bars, grp['Lift (%)']):
        ax.text(bar.get_x() + bar.get_width() / 2,
                v + (0.5 if v >= 0 else -1.5),
                f'{v:+.1f}%', ha='center', va='bottom', fontsize=9,
fontweight='bold')

plt.suptitle('Trial Period Performance Lift vs Scaled Control Store',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



8. Final Recommendations

Trial Store	Control Store	Sales Lift	Customer Lift	Driver	Verdict
77	233	**+26.2%**	**+23.1%**	Customer acquisition	**Proceed with monitored rollout**
86	155	**+13.2%**	**+13.5%**	Customer acquisition	**Phase rollout / Extend trial**
88	125	**+12.2%**	**+10.4%**	Acquisition + engagement	**Phase rollout / Extend trial**

Key Insights

- **Customer acquisition appears to be the main driver:** All three trial stores recorded higher customer numbers during the trial, suggesting that the new layout may have improved store appeal and encouraged more customers to visit and purchase.

- **Commercial lift vs. statistical significance:** Store 77 recorded the highest directional sales uplift (+26.2%). However, the three-month trial period provides a small sample for statistical testing, and none of the stores achieved statistical significance at the 5% level ($p > 0.05$). The results therefore indicate a positive commercial signal rather than conclusive evidence of a sustained effect.

- **Performance trend:** The positive results across all three trial stores are encouraging, but the limited trial period and high variation mean that the impact should be monitored over a longer period before drawing a definitive conclusion.

Recommendation to Julia

Proceed with a phased and monitored rollout, prioritising stores with demographic and customer profiles similar to Store 77. The three trial stores all showed positive directional improvements in sales and customer acquisition, with Store 77 demonstrating the strongest uplift. However, the three-month trial does not provide sufficient statistical evidence to confirm that the observed improvements will be sustained.

A phased implementation would allow the business to capture the potential commercial benefit while continuing to track customer acquisition, sales and engagement over a longer period. The additional data can then be used to assess whether the observed uplift becomes statistically significant and supports a wider rollout.